

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: KONA ANUSHA

Roll No.: A24126552088

Date: 29-08-2026

TASK 3 — JAVASCRIPT, DOM MANIPULATION AND STUDENT PROFILE

1. TITLE

Develop a Dynamic Student Profile Application Using JavaScript and DOM Manipulation

2. AIM

To develop a dynamic **Student Profile Application using JavaScript** that creates a Student object using user-provided values and dynamically displays the student details on a web page using **DOM manipulation**.

3. OBJECTIVE

The objectives of this task are:

- To understand JavaScript classes and objects.
- To create a `Student` class with appropriate properties.
- To initialize student details using a constructor.
- To accept student information from the user.
- To create a Student object using the entered values.
- To understand JavaScript event handling.
- To use DOM methods to access HTML elements.
- To dynamically create HTML elements using JavaScript.
- To display student details dynamically on the webpage.
- To apply CSS styling to the dynamically generated student profile.
- To perform basic form validation using JavaScript.

4. THEORY

JavaScript is a scripting language used to add dynamic behavior and interactivity to web pages. It can respond to user actions, manipulate HTML elements, validate input, and dynamically update webpage content.

In this application, a **JavaScript class** named `Student` is created. The class contains the following properties:

- **Name**
- **Roll Number**
- **Department**
- **CGPA**

A **constructor** is used to initialize these properties when a `Student` object is created.

The application also uses the **DOM (Document Object Model)** to access and modify HTML elements dynamically.

The basic working flow is:

User Input → Button Click → JavaScript Event → Student Object → DOM Manipulation → Student Profile Display

JavaScript Class

A class provides a blueprint for creating objects. The `Student` class is used to represent student information.

Constructor

The constructor initializes the properties of each `Student` object.

Event Listener

The `addEventListener()` method is used to detect when the **Display Student Profile** button is clicked.

DOM Manipulation

JavaScript DOM methods such as:

- `getElementById()`
- `createElement()`
- `appendChild()`

are used to access existing elements and dynamically create new elements.

Input Validation

Before creating the `Student` object, the program checks whether all required fields have been entered. If any field is empty, an alert message is displayed.

CSS Styling

CSS is used to style the main container, input fields, button, and dynamically generated student profile.

5. ALGORITHM / PROCEDURE

1. Create an HTML file containing the student input form.
2. Create input fields for Name, Roll Number, Department, and CGPA.
3. Add a button to display the student profile.

4. Create a JavaScript file and define a Student class.
 5. Define the constructor with Name, Roll Number, Department, and CGPA parameters.
 6. Assign the received values to the Student object properties.
 7. Access the display button using getElementById().
 8. Access the profile container using getElementById().
 9. Add a click event listener to the display button.
 10. Read the values entered by the user.
 11. Check whether all required fields are filled.
 12. Display an alert if any field is empty.
 13. Create a new object of the Student class using the entered values.
 14. Clear any previously displayed student profile.
 15. Dynamically create a <div> element for the profile.
 16. Dynamically create a heading and paragraph elements.
 17. Add the student details to the created elements.
 18. Append the created elements to the profile container.
 19. Display the generated student profile on the webpage.
 20. Apply CSS styling to the generated profile.
 21. Test the application with valid and invalid inputs.
 22. Verify the output with the expected student details.
-

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile</title>
  <!-- Link external CSS file -->
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Student Registration</h1>
    <!-- Student input form -->
    <label for="name">Name:</label>
    <input type="text" id="name" placeholder="Enter student name">
    <label for="rollNo">Roll Number:</label>
    <input type="text" id="rollNo" placeholder="Enter roll number">
    <label for="department">Department:</label>
    <input type="text" id="department" placeholder="Enter department">
```

```
<label for="cgpa">CGPA:</label>
<input type="number" id="cgpa" step="0.01" placeholder="Enter CGPA">
<!-- Button to generate the profile -->
<button id="displayBtn">Display Student Profile</button>
<!-- Dynamically generated profile will appear here -->
<div id="profileContainer"></div>
</div>
<!-- Link external JavaScript file -->
<script src="script.js"></script>
</body>
</html>
```

script.js

```
// Create Student class
class Student {
  // Constructor initializes student properties
  constructor(name, rollNo, department, cgpa) {
    this.name = name;
    this.rollNo = rollNo;
    this.department = department;
    this.cgpa = cgpa;
  }
}

// Select the button and profile container
const displayButton = document.getElementById("displayBtn");
const profileContainer = document.getElementById("profileContainer");
// Add click event to the button
displayButton.addEventListener("click", function () {
  // Get values entered by the user
  const name = document.getElementById("name").value;
  const rollNo = document.getElementById("rollNo").value;
  const department = document.getElementById("department").value;
  const cgpa = document.getElementById("cgpa").value;
  // Check whether all fields are filled
  if (name === "" || rollNo === "" ||
    department === "" || cgpa === "") {
    alert("Please enter all student details.");
    return;
  }
  // Create an object of Student class
  const student = new Student(
    name,
    rollNo,
    department,
```

```
        cgpa
    );
    // Remove previously displayed profile
    profileContainer.innerHTML = "";
    // Create profile container dynamically
    const profile = document.createElement("div");
    profile.className = "profile";
    // Create heading
    const heading = document.createElement("h2");
    heading.textContent = "Student Profile";
    // Create student detail elements
    const namePara = document.createElement("p");
    namePara.textContent = "Name : " + student.name;
    const rollPara = document.createElement("p");
    rollPara.textContent = "Roll No : " + student.rollNo;
    const departmentPara = document.createElement("p");
    departmentPara.textContent =
        "Department : " + student.department;
    const cgpaPara = document.createElement("p");
    cgpaPara.textContent = "CGPA : " + student.cgpa;
    // Add elements to the profile
    profile.appendChild(heading);
    profile.appendChild(namePara);
    profile.appendChild(rollPara);
    profile.appendChild(departmentPara);
    profile.appendChild(cgpaPara);
    // Display the profile on the webpage
    profileContainer.appendChild(profile);
});
```

style.css

```
* {
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background-color: #a1939342;
    margin: 0;
    padding: 30px;
}

/* Main container */
.container {
    width: 450px;
    margin: auto;
    padding: 25px;
```

```
background-color: rgb(98, 123, 124);
border-radius: 10px;
box-shadow: 0 4px 12px rgb(54, 104, 73);
}
```

```
/* Page heading */
h1 {
  text-align: center;
  color: #070707;
}
```

```
/* Form labels */
label {
  display: block;
  margin-top: 12px;
  font-weight: bold;
}
```

```
/* Input fields */
input {
  width: 100%;
  padding: 10px;
  margin-top: 5px;
  border: 1px solid #000000;
  border-radius: 5px;
}
```

```
/* Display button */
button {
  width: 100%;
  padding: 12px;
  margin-top: 20px;
  background-color: #1a1224;
  color: rgb(239, 240, 239);
  border: none;
  border-radius: 5px;
  font-size: 16px;
  cursor: pointer;
}
```

```
/* Button hover effect */
button:hover {
  background-color: #306ba9;
}
```

```
/* Dynamically generated student profile */
.profile {
```

```

margin-top: 25px;
padding: 20px;
background-color: #f2ebf3;
border-left: 5px solid #3d303d;
border-radius: 8px;
}

/* Profile heading */
.profile h2 {
margin-top: 0;
text-align: center;
color: #551c45;
}

/* Profile details */
.profile p {
font-size: 16px;
line-height: 1.6;
}

```

7. CODE EXPLANATION

Code	Explanation
class Student	Defines a class named Student.
constructor()	Initializes the properties of a Student object.
this.name	Stores the student's name.
this.rollNo	Stores the student's roll number.
this.department	Stores the student's department.
this.cgpa	Stores the student's CGPA.
getElementById()	Selects an HTML element using its ID.
addEventListener()	Detects and responds to user events such as button clicks.
.value	Retrieves the value entered in an input field.
alert()	Displays a message to the user.
new Student()	Creates an object of the Student class.
innerHTML = ""	Clears previously generated content.
createElement()	Dynamically creates a new HTML element.
className	Assigns a CSS class to an element.
textContent	Sets the text content of an HTML element.
appendChild()	Adds a dynamically created element to another element.
.profile	CSS class used to style the generated student profile.
button: hover	Applies styling when the mouse pointer is over the button.

8. TEST CASES AND EXECUTION DATA

Valid Input Test Cases

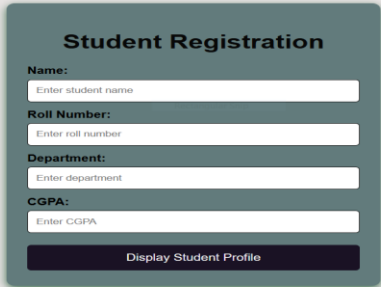
Test Case	Name	Roll No.	Department	CGPA	Expected Result	Actual Result	Status
TC-01	Ravi Kumar	A23126552001	CSE (AI & ML)	8.42	Student profile should be displayed	Student profile displayed correctly	Pass
TC-02	Priya	A23126552002	CSE	9.10	Student profile should be displayed	Student profile displayed correctly	Pass
TC-03	Arjun	A23126552003	ECE	7.85	Student profile should be displayed	Student profile displayed correctly	Pass

Validation Test Case

Test Case	Input	Expected Result	Actual Result	Status
TC-04	One or more fields left empty	Alert should display "Please enter all student details."	Alert displayed correctly	Pass

9. OUTPUT

Output 1: Student Information Input Page

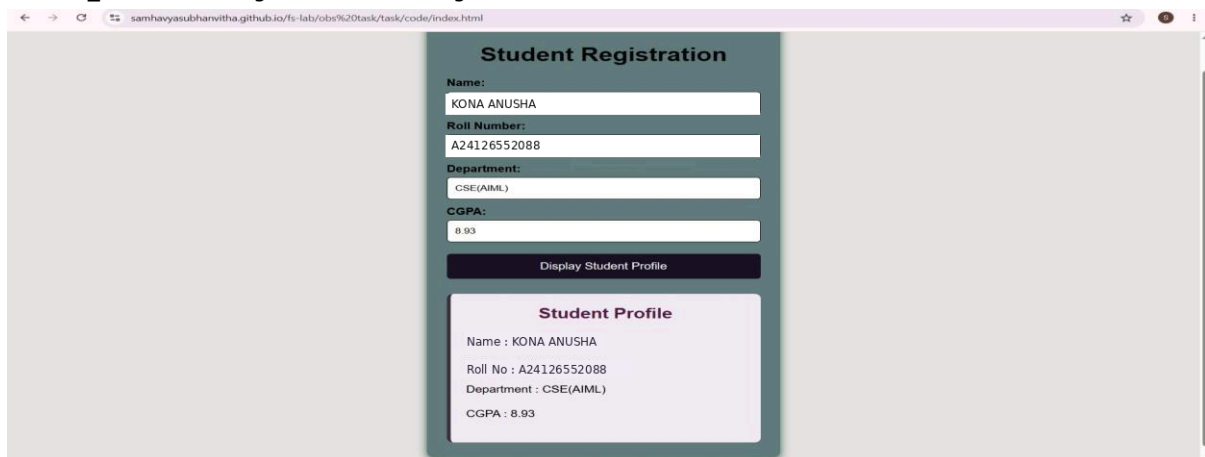


The screenshot shows a 'Student Registration' form with the following fields and a button:

- Name:** Enter student name
- Roll Number:** Enter roll number
- Department:** Enter department
- CGPA:** Enter CGPA
- Display Student Profile** (button)

Figure 1: Student Profile application before displaying the student details.

Output 2: Dynamically Generated Student Profile



The screenshot shows a web browser window with a URL bar containing 'samhavyasubharvitha.github.io/fs-lab/obs%20task/task/code/index.html'. The main content area displays a 'Student Registration' form with the following fields: Name (KONA ANUSHA), Roll Number (A24126552088), Department (CSE(AIML)), and CGPA (8.93). Below the form is a 'Display Student Profile' button. To the right of the button, a 'Student Profile' box is displayed, showing the same information: Name : KONA ANUSHA, Roll No : A24126552088, Department : CSE(AIML), and CGPA : 8.93.

Figure 2: Student Profile dynamically generated using JavaScript DOM manipulation.

10. INTERVIEW QUESTIONS

Q1. What is a JavaScript class?

Answer:

A JavaScript class is a blueprint for creating objects. It defines the properties and methods that objects created from the class can have.

Q2. What is the purpose of a constructor in a JavaScript class?

Answer:

A constructor is a special method that is automatically called when an object is created. It is used to initialize the properties of the object.

Q3. What is DOM manipulation?

Answer:

DOM manipulation is the process of using JavaScript to access, create, modify, or remove HTML elements dynamically in a web page.

Q4. What is the purpose of createElement()?

Answer:

The createElement() method creates a new HTML element dynamically using JavaScript. In this application, it is used to create the student profile and its contents.

Q5. What is the purpose of appendChild()?

Answer:

The appendChild() method adds a newly created HTML element as a child of an existing element.

Q6. Why is addEventListener() used in this application?

Answer:

addEventListener() is used to detect the button click event and execute the code that creates and displays the student profile.

Q7. What happens if one of the input fields is empty?

Answer:

The program checks whether any required field is empty. If a field is empty, an alert message **"Please enter all student details."** is displayed and the Student object is not created.

11. RESULT

Thus, a **Dynamic Student Profile Application** was successfully developed using **JavaScript classes, objects, event handling, and DOM manipulation**. Student details were accepted from the user, a Student object was created dynamically, and the student profile was generated and displayed on the webpage with CSS styling. The application was successfully tested using valid and invalid inputs.